



3.4 只读存储器

3.4.1 只读存储器ROM

3.4.2 NOR闪存

3.4.1 只读存储器ROM

ROM叫做只读存储器。

只读的意思是在它工作时只能读出，不能写入。

然而其中存储的原始数据，必须在它工作以前写入。

只读存储器由于工作可靠，保密性强，在计算机系统中得到广泛的应用。

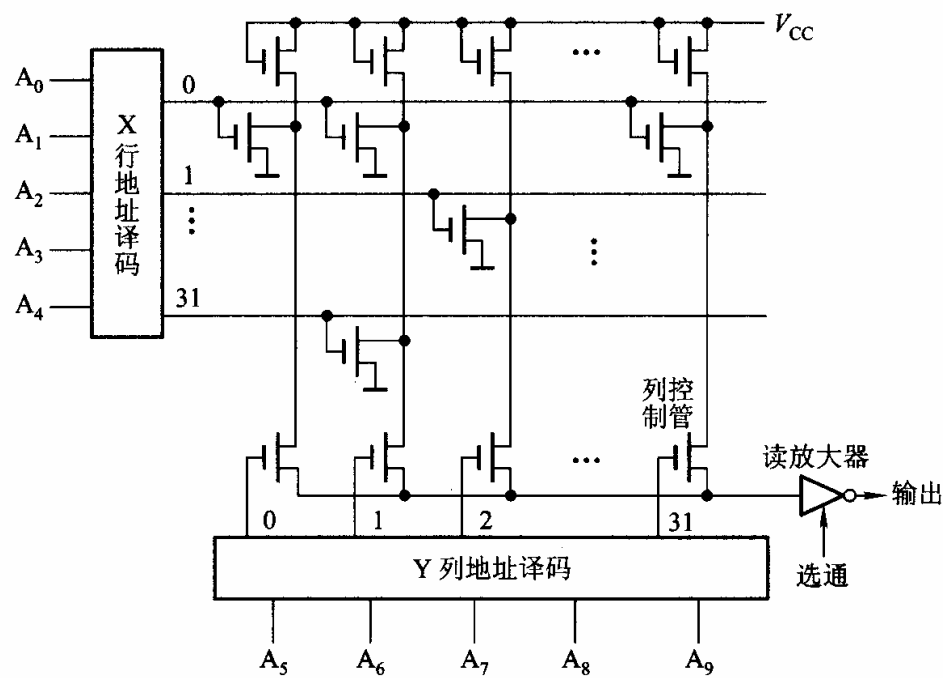
主要有两类：

- 掩模ROM：掩模ROM实际上是一个存储内容固定的ROM，由生产厂家提供产品。
- 可编程ROM：用户后写入内容，有些可以多次写入。
 - 一次性编程的PROM
 - 多次编程的EPROM、E²PROM和闪速存储器(闪存)。

3.4.1 只读存储器ROM

1. 掩模ROM

(1) 掩模ROM

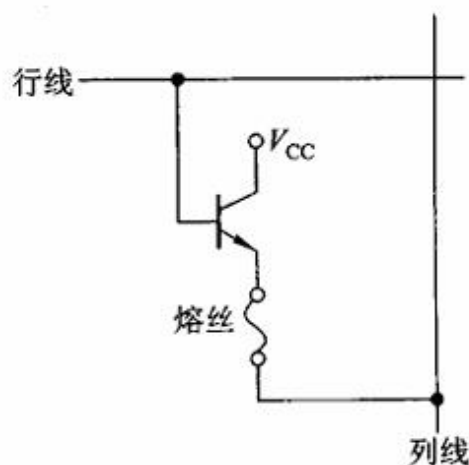


1K×1位的MOS管掩膜

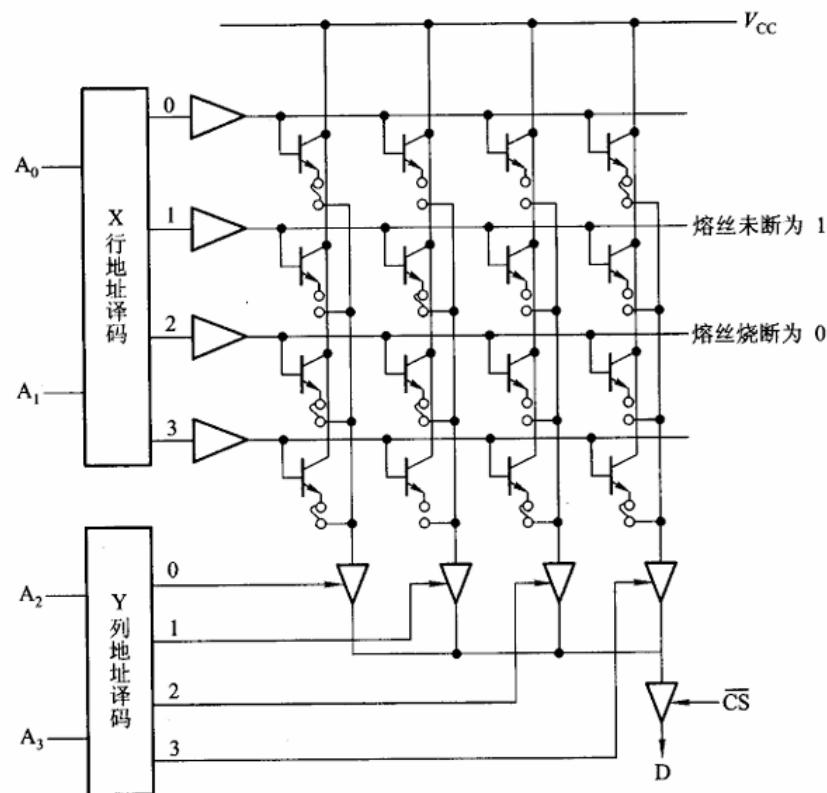
3.4.1 只读存储器ROM

2.可编程ROM

- PROM—一次性编程的只读存储器。



双极型镍铬熔丝
式单元电路



16 × 1 位双极型镍铬熔丝式 PROM

3.4.1 只读存储器ROM

• 2、可编程ROM

EPROM存储元 叫做可擦除可编程只读存储器。

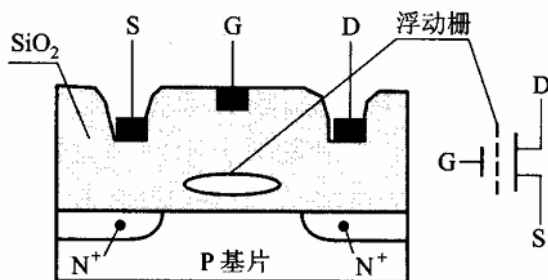
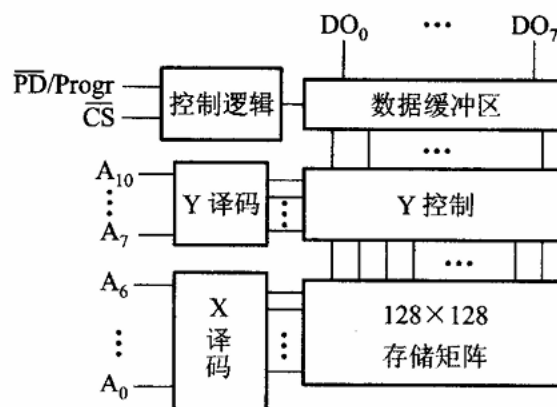
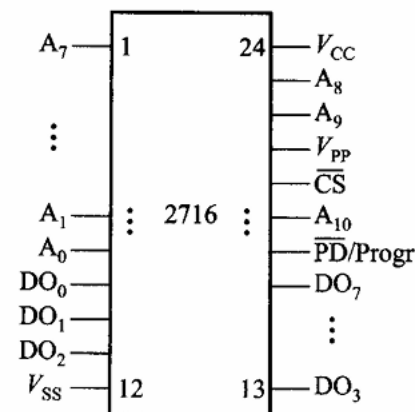


图 4.30 N型沟道浮动栅 MOS 电路



(a) 逻辑图



(b) 引脚图

图 4.31 2716 型 EPROM 逻辑图及引脚图

EPROM又分为紫外线擦除EPROM(UV-EPROM)、电擦除EPROM(EEPROM或E2PROM)和闪速(Flash)只读存储器

3.4.1 只读存储器ROM

2.可编程ROM

- FLASH存储器--闪速存储器，
- 是高密度非易失性的读/写存储器。
- 高密度意味着它具有巨大比特数目的存储容量。
- 非易失性意味着存放的数据在没有电源的情况下可以长期保存。
- 闪速存储器(闪存)也属于电可擦、可在线编程的非易失性只读存储器。闪速存储器的存储密度高，工作速度快，擦除块尺寸较大
- 可以分为 NOR 技术、DINOR 技术、AND技术和 NAND 技术等不同类别。

3.4.1 只读存储器ROM

表 3.3 比较了常见的各类存储器的主要特征。

表 3.3 存储器特性比较

存储器类型	非易失性	高密度	低功耗	可在线更新	快速读出
Flash 闪存	√	√	√	√	√
SRAM	—	—	—	√	√
DRAM	—	√	—	√	√
EEPROM	√	—	√	√	√
OTP ROM	√	√	√	—	√
UV-EPROM	√	√	√	—	√
掩模 ROM	√	√	√	—	√
硬盘	√	√	—	√	—
CD-ROM 光盘	√	√	—	—	—

3.4.2 NOR内存

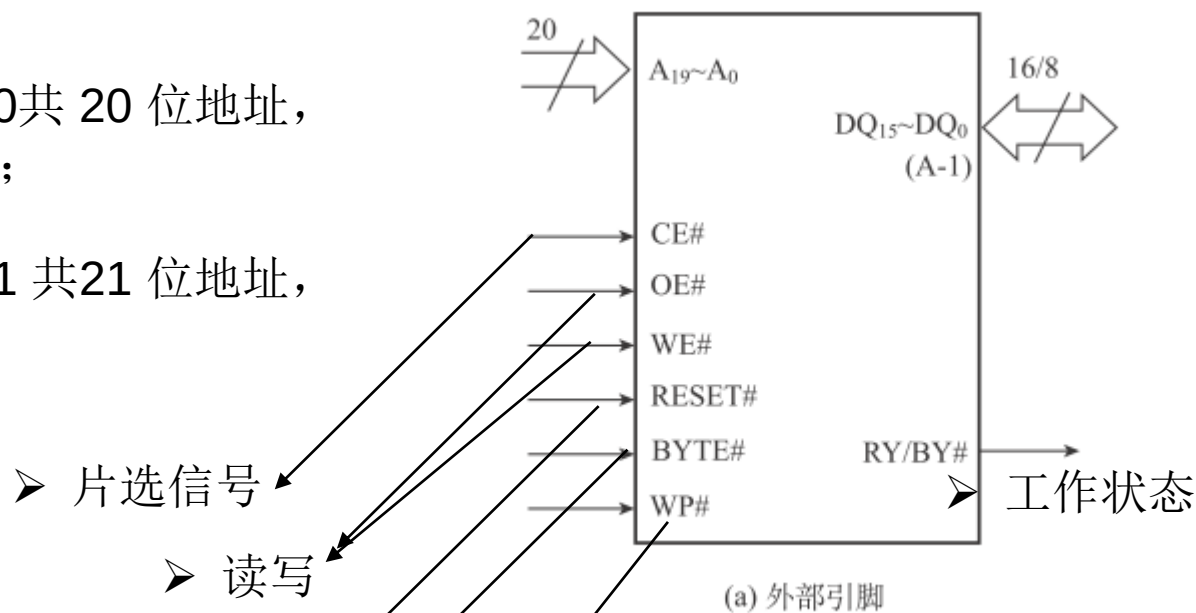
1. NOR 闪存的外部接口与逻辑结构

➤ 字模式

1M×16bit, 需要 A19~A0共 20 位地址,
DQ15~DQ0共 16 位数据线;

➤ 字节模式

2M×8bit, 需要A19~A-1 共21 位地址,
DQ7~DQ0共8位数据线。

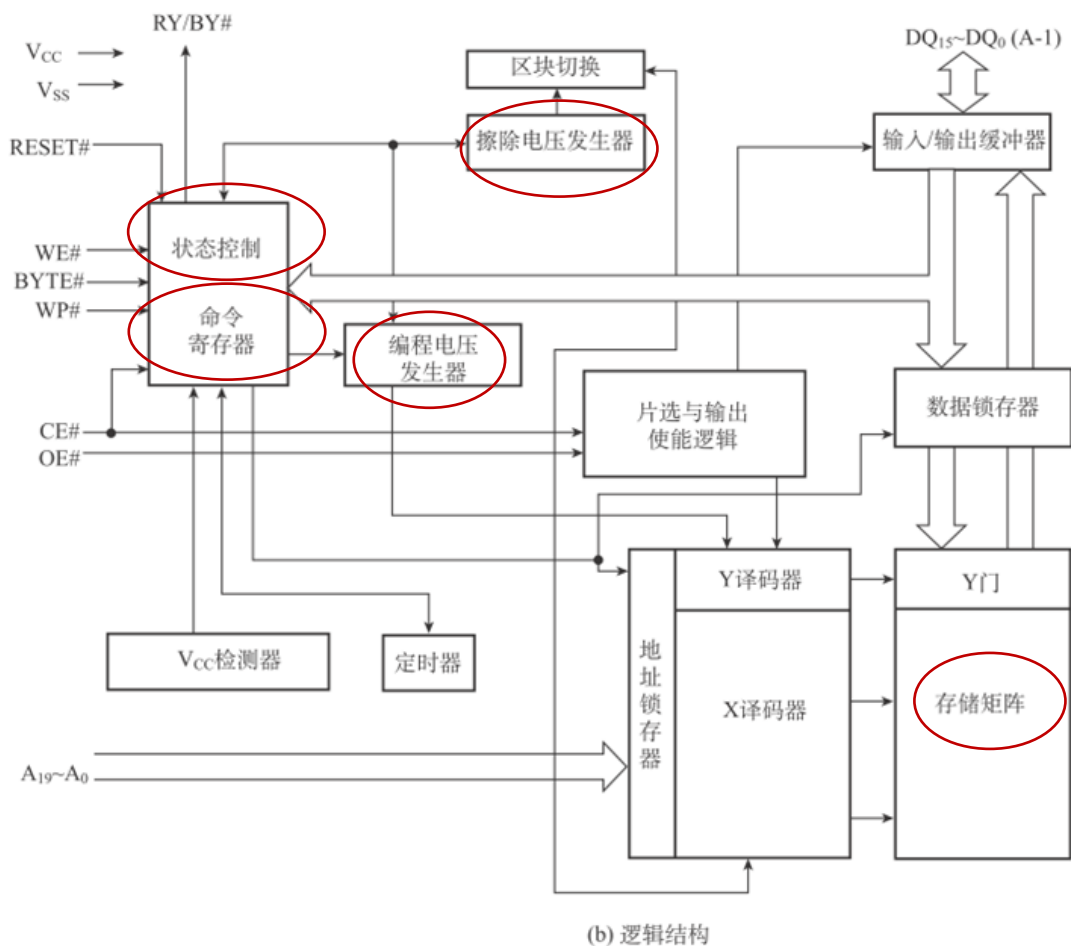


➤ 低电平有效, 低--字节模式; 高一字模式

➤ 写保护状态

3.4.2 NOR内存

1. NOR 闪存的外部接口与逻辑结构



3.4.2 NOR内存

2. NOR 闪存的区块划分

- S29AL016J 是一种区块(sector)式闪存，外部按 $1\text{M} \times 16\text{bit}$ 或 $2\text{M} \times 8\text{bit}$ 组织，内部组织为 35 个区块。书中的表 3.4 给出了底部引导区版本的区块地址表，其低地址区通常存放系统引导程序和一些参数，因而区块尺寸较小并且可以附加特定的写保护措施，前 4 个区块大小分别为 16KB、8KB、8KB、32KB。SA4~SA34 大小均为 64KB。每个存储单元的地址由高位的区块地址(A_{12} 以上)和低位的区块内偏移地址两个字段组成，两个字段的长度与区块尺寸相关。

3.4.2 NOR内存

3. NOR 闪存的总线操作与工作方式

表 3.5 S29AL016J 的部分总线操作

操作	CE#	OE#	WE#	RESET#	WP#	地址	DQ ₀ ~DQ ₇	DQ ₈ ~DQ ₁₅	
								BYTE# = 0	BYTE# = 1
读	0	0	1	1	任意	地址输入	数据输出	数据输出	DQ ₈ ~DQ ₁₄ = 高阻
写	0	1	0	1	*	地址输入	数据输入/输出	数据输入/输出	DQ ₁₅ = A ₋₁
保持	V _{CC} ±0.3V	任意	任意	V _{CC} ±0.3V	任意	任意	高阻	高阻	高阻
输出禁止	0	1	1	1	任意	任意	高阻	高阻	高阻
复位	任意	任意	任意	0	任意	任意	高阻	高阻	高阻

*WP#=0 时，最外区块保持保护状态；WP#=1 时，最外区块的保护状态由先前的保护/去保护状态决定。

除了读出和编程写入这些常规的 **PROM** 操作外，NOR 闪存还具有内部控制寄存器和状态寄存器，可以通过“命令写”和“状态读”操作进行灵活的控制。闪存内部通过状态机控制其操作状态。

3.4.2 NOR内存

3. NOR 闪存的总线操作与工作方式

表 3.6 S29AL016J 的部分命令定义(字模式)

命令序列	总线周期数	总线周期											
		周期 1		周期 2		周期 3		周期 4		周期 5		周期 6	
		地址	数据	地址	数据	地址	数据	地址	数据	地址	数据	地址	数据
复位	1	×××	F0										
读	1	读地址	读数据										
芯片擦除	6	555	AA	2AA	55	555	80	555	AA	2AA	55	555	10
区块擦除	6	555	AA	2AA	55	555	80	555	AA	2AA	55	区块地址	30
编程	4	555	AA	2AA	55	555	A0	编程地址	编程数据				

从 NOR 闪存的编程和擦除方式可以看出闪存与 RAM 的差异：

- 闪存的存储单元在编程之前需首先擦除；闪存发出编程命令比 RAM 复杂；
- 闪存编程的速度远低于 RAM 的写入速度；
- 闪存的读出速度也远低于 RAM。

可见，虽然闪存具有非易失性并可在线编程，但仍然属于 ROM，一般情况下闪存不能取代 RAM。

3.5 并行存储器

3.5.1 双端口存储器（空间并行技术）

3.5.2 多模块交叉存储器（时间并行技术）

3.5 并行存储器

CPU和主存储器之间速度不匹配，限制了高速计算机设计。

为了提高CPU和主存之间的数据交换速率，可以在不同层次采用不同的技术加速存储器访问速度。

3.5 并行存储器

- 芯片技术

提高单个芯片的访问速度。可以选用更高速的半导体器件，或者改善存储芯片内部结构和对外接口方式。

e.g.突发传输技术、同步DRAM 技术和CDRAM 技术等。

- 结构技术

为了解决存储器与CPU速度不匹配问题，需要改进存储器与CPU之间的连接方式，加速CPU和存储器之间的有效传输。

e.g.采用并行技术的双口存储器甚至是多口存储器，以及多体交叉存储器，都可以让CPU在一个周期中访问多个存储字。

- 系统结构技术

从整个存储系统的角度采用分层存储结构解决访问速度问题

e.g.增加cache，采用虚拟存储器等。

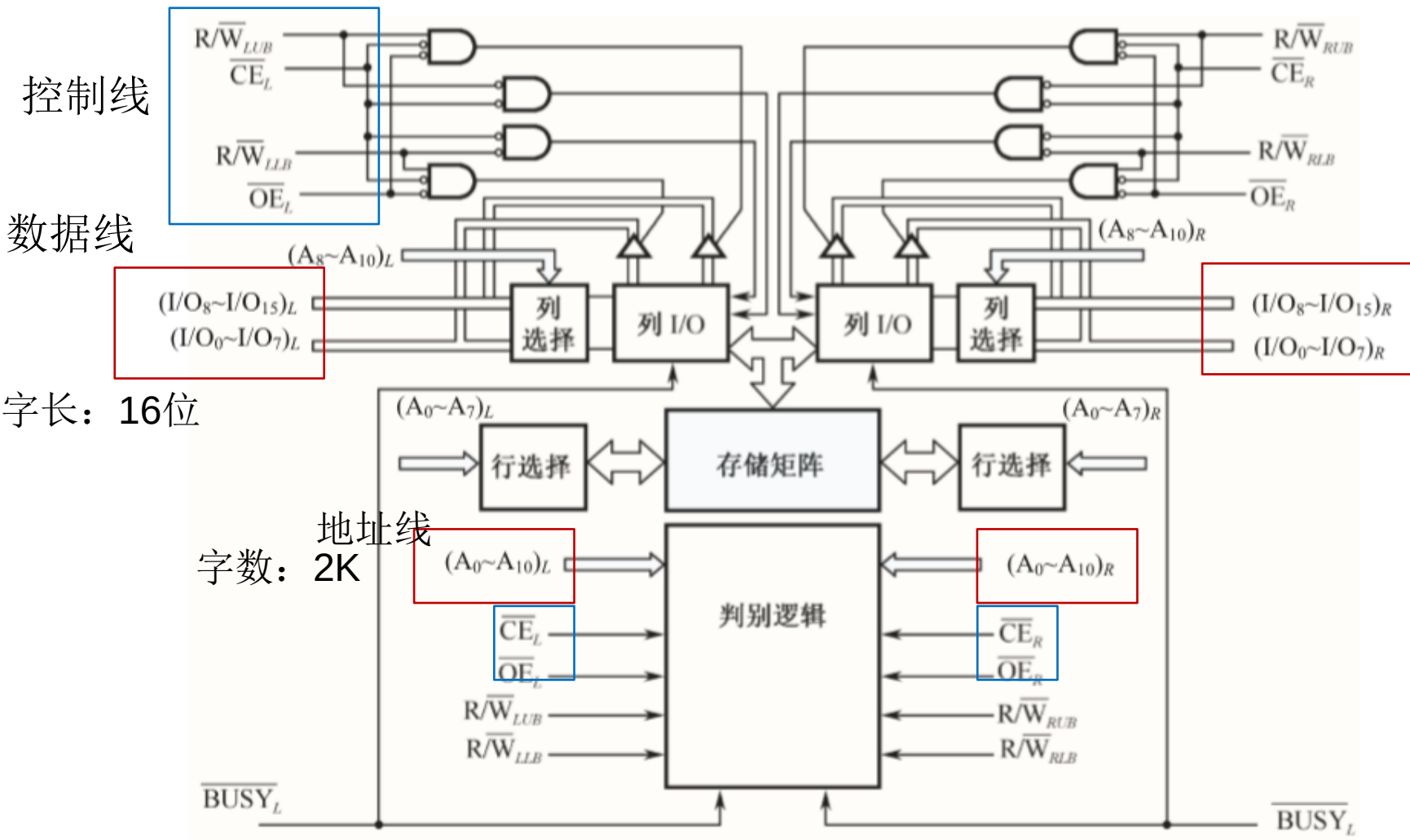
3.5.1 双端口存储器

1. 双端口存储器的逻辑结构

- 同一个存储器具有**两组相互独立的读写控制电路**
- 举例说明，双端口存储器IDT7133的逻辑框图**SRAM**。
- 事实上双端口存储器也可以由**DRAM**构成。

3.5.1 双端口存储器

举例说明，双端口存储器IDT7133的逻辑框图。



3.5.1 双端口存储器

2. 无冲突读写控制

当两个端口的地址不相同时，在两个端口上进行读写操作，一定不会发生冲突。当任一端口被选中驱动时，就可对整个存储器进行存取，每一个端口都有自己的片选控制($\overline{CE}$)和输出驱动控制($\overline{OE}$)。读操作时，端口的 $\overline{OE}$ (低电平有效)打开输出驱动器，由存储矩阵读出的数据就出现在I/O线上。

3.5.1 双端口存储器

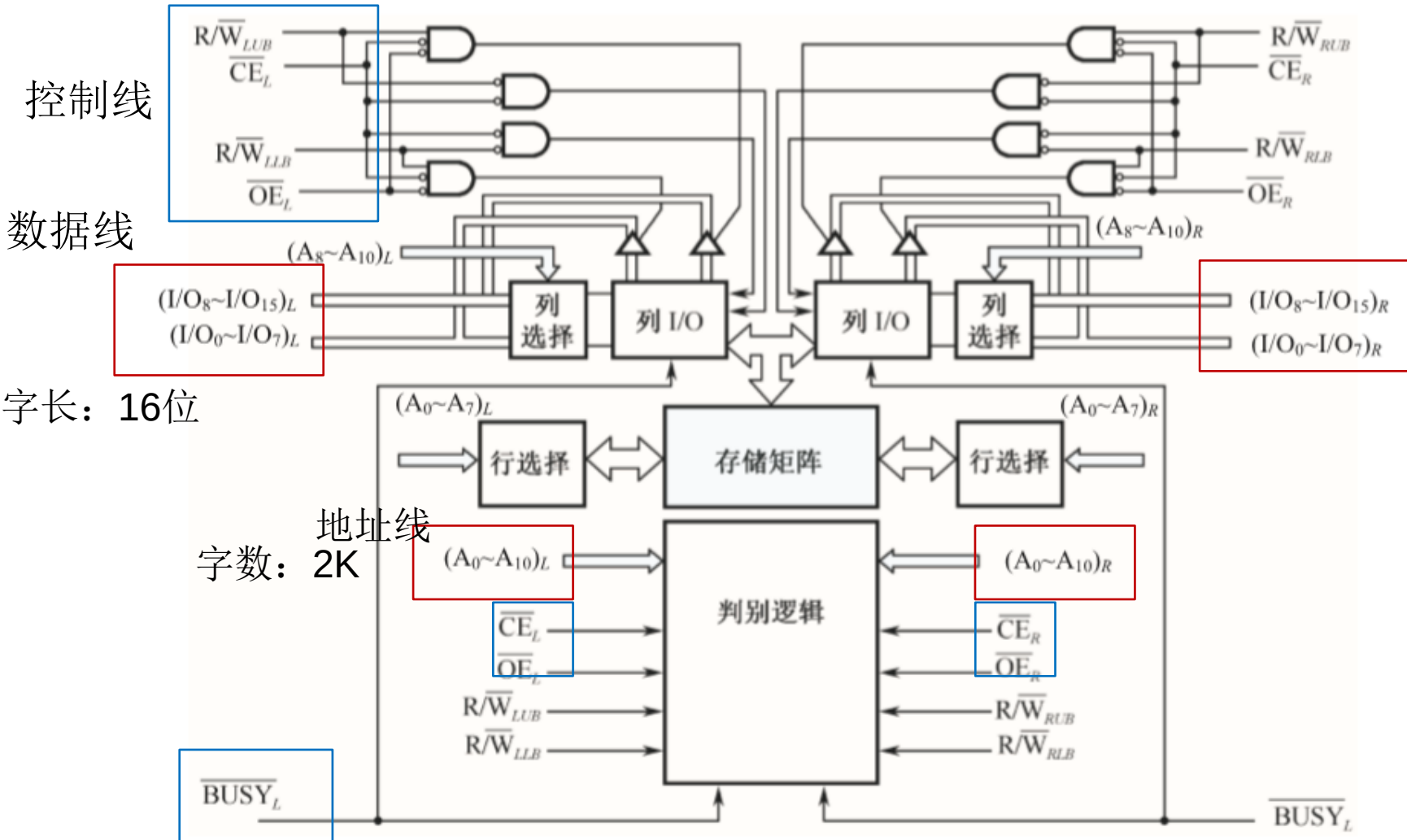
2.无冲突读写控制

左端口或右端口						功 能
$R/\overline{W}_{LB}$	$R/\overline{W}_{UB}$	$\overline{CE}$	$\overline{OE}$	$I/O_0 \sim I/O_7$	$I/O_8 \sim I/O_{15}$	
×	×	1	1	Z	Z	端口不用
0	0	0	×	数据入	数据入	低位和高位字节数据写入存储器($\overline{BUSY}$ 高电平)
0	1	0	0	数据入	数据出	低位字节数据写入存储器, 存储器中数据输出至高位字节
1	0	0	0	数据出	数据入	存储器中数据输出至低位字节, 高位字节数据写入存储器
0	1	0	1	数据入	Z	低位字节数据写入存储器
1	0	0	1	Z	数据入	高位字节数据写入存储器
1	1	0	0	数据出	数据出	存储器中数据输出至低位字节和高位字节
1	1	0	1	Z	Z	高阻抗输出

上表列出了无冲突的读写条件, 表中符号 **1** 代表高电平, **0** 为低电平, **×** 为任意, **Z** 为高阻态。

3.5.1 双端口存储器

举例说明，双端口存储器IDT7133的逻辑框图。



3.5.1 双端口存储器

3. 有冲突读写控制

当两个端口同时存取存储器同一存储单元时，而且至少有一个端口为写操作时，便发生读写冲突。为解决此问题，特设置了 *BUSY* 标志。在这种情况下，片上的判断逻辑可以决定对哪个端口优先进行读写操作，而对另一个被延迟的端口置 *BUSY* 标志(*BUSY*变为低电平)，即暂时关闭此端口。

3.5.1 双端口存储器

有冲突读写控制判断方法

- (1)如果地址匹配且在 $\overline{CE}$ 之前有效，片上的控制逻辑在 $\overline{CE}_L$ 和 $\overline{CE}_R$ 之间进行判断来选择端口($\overline{CE}$ 判断)。
- (2)如果 $\overline{CE}$ 在地址匹配之前变低，片上的控制逻辑在左、右地址间进行判断来选择端口(地址有效判断)。

无论采用哪种判断方式，延迟端口的 $\overline{BUSY}$ 标志都将置位而关闭此端口，而当允许存取的端口完成操作时，延迟端口 $\overline{BUSY}$ 标志才进行复位而打开此端口。

3.5.1 双端口存储器

表 3.8 左、右端口读写操作的功能判断

左端口		右端口		标志		功能	说明
$\overline{CE}_L$	$(A_0 \sim A_{10})_L$	$\overline{CE}_R$	$(A_0 \sim A_{10})_R$	$\overline{BUSY}_L$	$\overline{BUSY}_R$		
1	×	1	×	1	1	无冲突	$\overline{CE}$ 在地址匹配之前 变低的地址判断
0	任意	1	×	1	1	无冲突	
1	×	0	任意	1	1	无冲突	
0	$\neq (A_0 \sim A_{10})_R$	0	$\neq (A_0 \sim A_{10})_L$	1	1	无冲突	
0	$LV5R$	0	$LV5R$	1	0	左端口取胜	
0	$RV5L$	0	$RV5L$	0	1	右端口取胜	
0	相同	0	相同	1	0	消除判断	
0	相同	0	相同	0	1	消除判断	
$LL5R$	$= (A_0 \sim A_{10})_R$	$LL5R$	$= (A_0 \sim A_{10})_L$	1	0	左端口取胜	地址匹配在 $\overline{CE}$ 之前 的 $\overline{CE}$ 判断
$RL5L$	$= (A_0 \sim A_{10})_R$	$RL5L$	$= (A_0 \sim A_{10})_L$	0	1	右端口取胜	
$LW5R$	$= (A_0 \sim A_{10})_R$	$LW5R$	$= (A_0 \sim A_{10})_L$	1	0	消除判断	
$LW5R$	$= (A_0 \sim A_{10})_R$	$LW5R$	$= (A_0 \sim A_{10})_L$	0	1	消除判断	

表中符号意义如下：

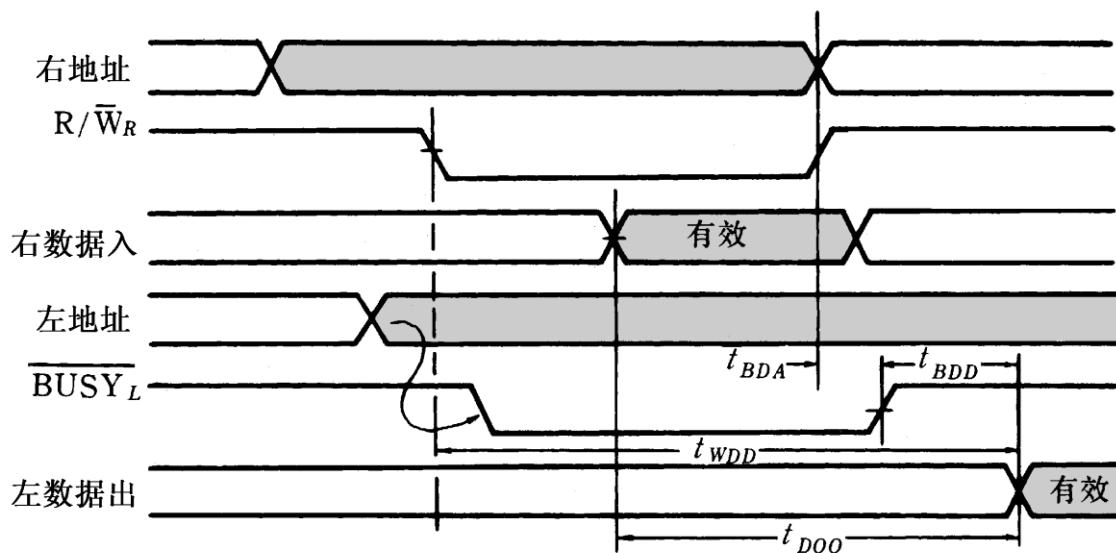
$LV5R$ ：左地址有效先于右地址 50ns，

$LL5R$ ： $\overline{CE}_L$ 变低先于 $\overline{CE}_R$ 50ns

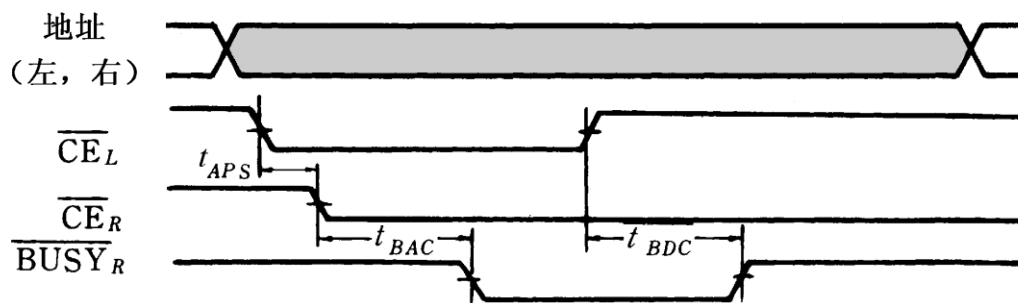
$RV5L$ ：右地址有效先于左地址 50ns，

$RL5L$ ： $\overline{CE}_R$ 变低先于 $\overline{CE}_L$ 50ns

3.5.1 双端口存储器



(a) 未发生冲突时读写时序波形 (左读右写)

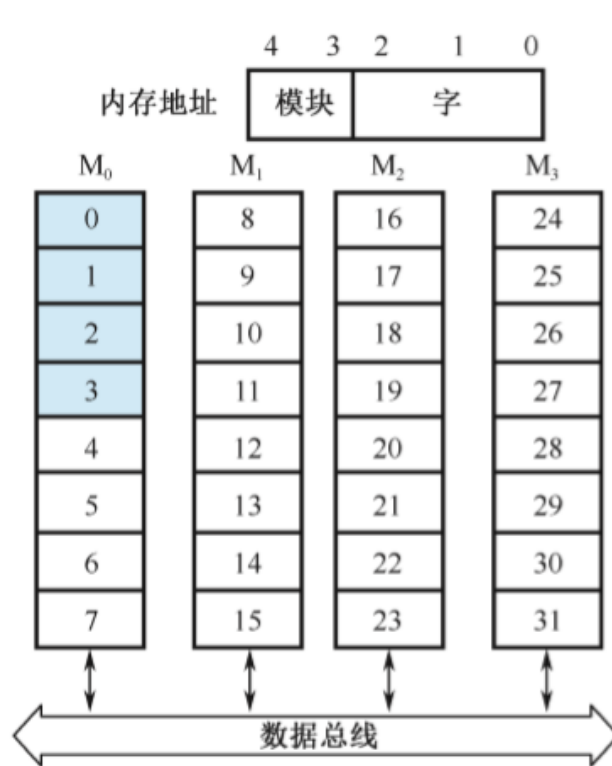


(b) 用 $\overline{CE}$ 判断的冲突周期时序波形 ($\overline{CE}_L$ 先有效)

3.5.2 多模块交叉存储器

1. 存储器的模块化组织

一个由若干个模块组成的主存储器是线性编址的。这些地址在各模块中的安排有两种方式：一种是顺序方式



(a) 顺序方式

3.5.2 多模块交叉存储器

1) 顺序方式

[例]M0—M3共四个模块，若采用顺序方式，则每个模块8个字

M0: 0—7

M1: 8—15

M2: 16—23

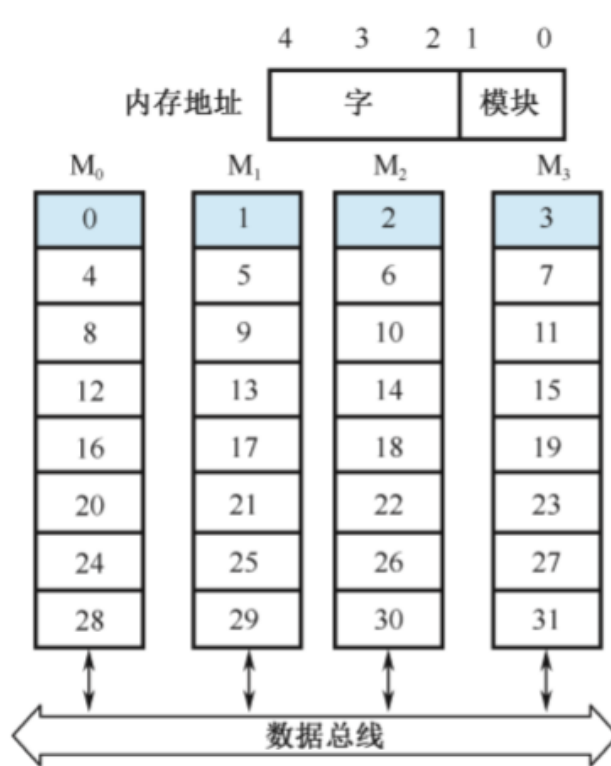
M3: 24—31

- 高位选模块，低位选块内地址
- 优点是：某个模块进行存取时，其他模块不工作，某一模块出现故障时，其他模块可以照常工作，通过增添模块来扩充存储器容量比较方便。
- 缺点是各模块串行工作，存储器的带宽受到了限制。

3.5.2 多模块交叉存储器

1. 存储器的模块化组织

一个由若干个模块组成的主存储器是线性编址的。这些地址在各模块中的安排有两种方式：一种是交叉方式



(b) 交叉方式

3.5.2 多模块交叉存储器

2) 交叉方式

[例]M0—M3共四个模块，若采用交叉方式，则每个模块8个字

M0: 0, 4, ... 除以4余数为0

M1: 1, 5, ... 除以4余数为1

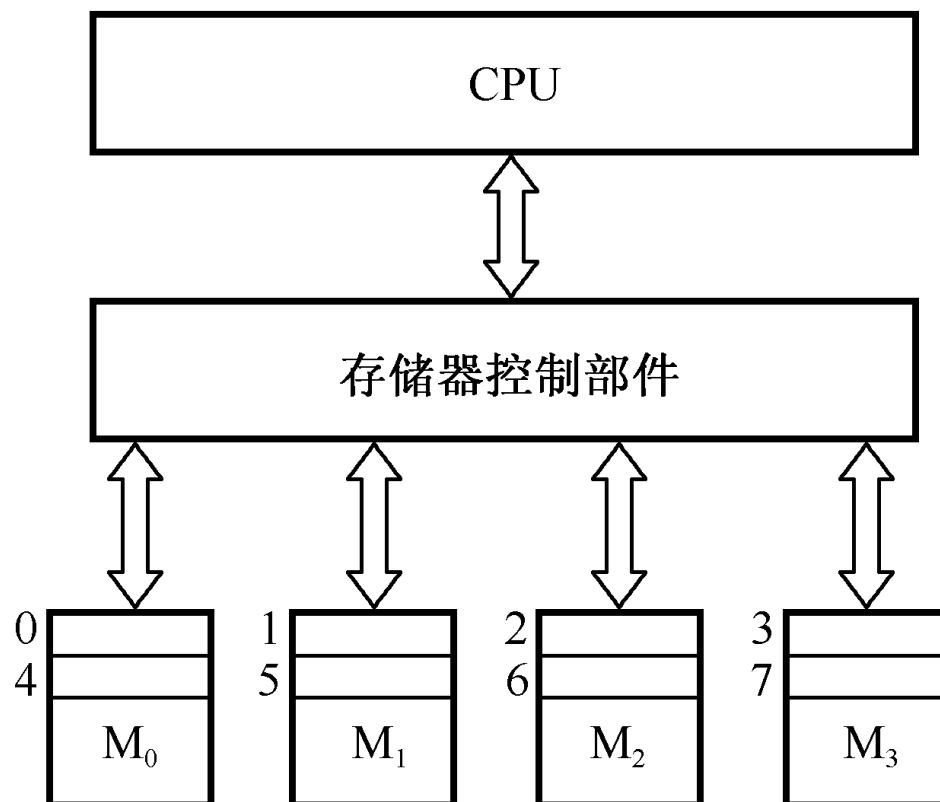
M2: 2, 6, ... 除以4余数为2

M3: 3, 7, ... 除以4余数为3

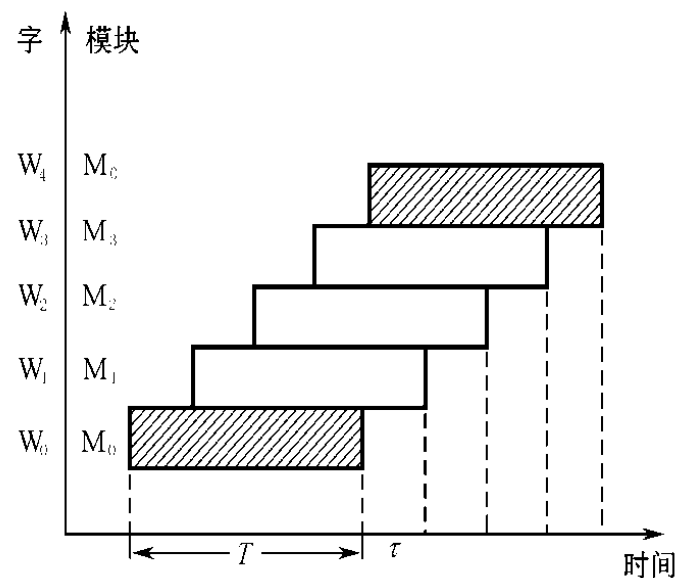
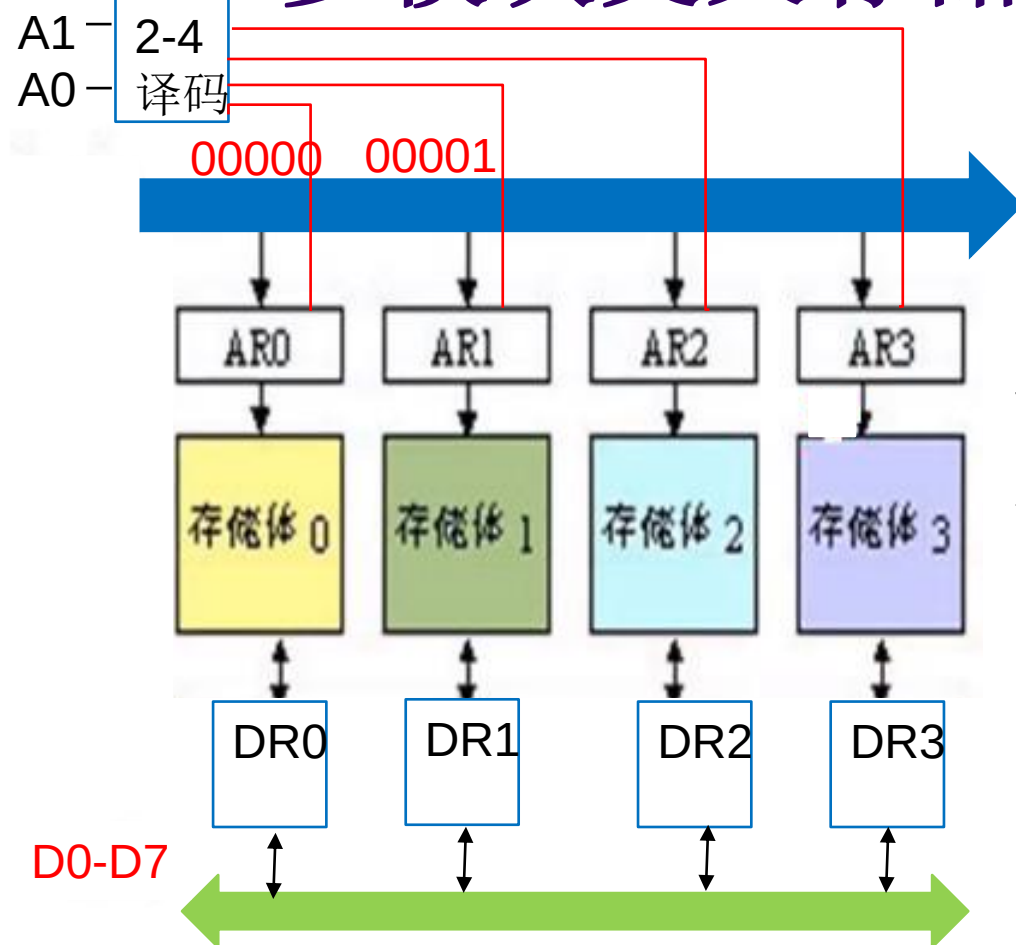
- 高位选块内地址，低位选模块
- 特点：连续地址分布在相邻的不同模块内，同一个模块内的地址都是不连续的。
- 优点是对连续字的成块传送可实现多模块流水式并行存取，大大提高存储器的带宽。

3.5.2 多模块交叉存储器

2. 多模块交叉存储器的基本结构



多模块交叉存储器



3.5.2 多模块交叉存储器

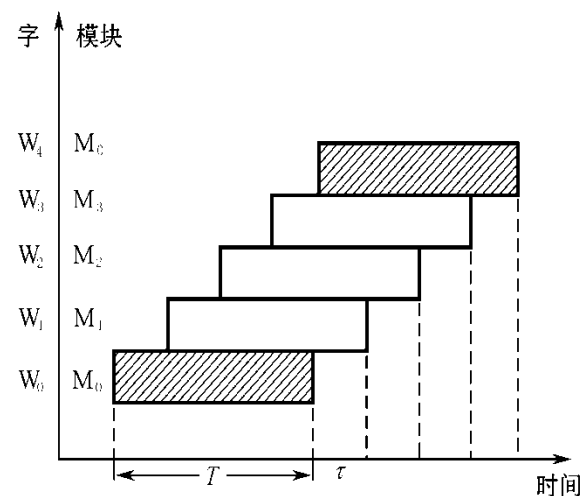
- 设模块字长等于数据总线宽度，模块存取一个字的存储周期为 T ，总线传送周期为 τ ，存储器的交叉模块数为 m ，那么为了实现流水线方式存取，应当满足 $T \leq m\tau$ ，即成块传送可按 τ 间隔流水方式进行，也就是每经 τ 时间延迟后启动下一个模块。
- m 的最小值 $m_{\min}$ 称为交叉存取度。交叉存储器要求其模块数必须大于或等于 $m_{\min}$ ，以保证启动某模块后经 $m\tau$ 时间再次启动该模块时，它的上次存取操作已经完成。这样，连续读取 m 个字所需的时间为

$$t_1 = T + (m-1)\tau$$

- 而顺序方式存储器连续读取 m 个字所需时间为

$$t_2 = mT$$

由于 $t_1 < t_2$ ，交叉存储器的带宽确实大大提高了。



例5 设存储器容量为**32字**，字长**64位**，模块数**m=4**，分别用顺序方式和交叉方式进行组织。存储周期**T=200ns**，数据总线宽度为**64位**，总线传送周期=**50ns**。若连续读出**4个字**，问顺序存储器和交叉存储器的带宽各是多少？

解：顺序存储器和交叉存储器连续读出**m=4**个字的信息总量都是：

$$q=64b \times 4=256b$$

顺序存储器和交叉存储器连续读出4个字所需的时间分别是：

$$t_2 = mT = 4 \times 200ns = 800ns = 8 \times 10^{-7}s$$

$$t_1 = T + (m-1)\tau = 200ns + 3 \times 50ns = 350ns = 35 \times 10^{-7}s$$

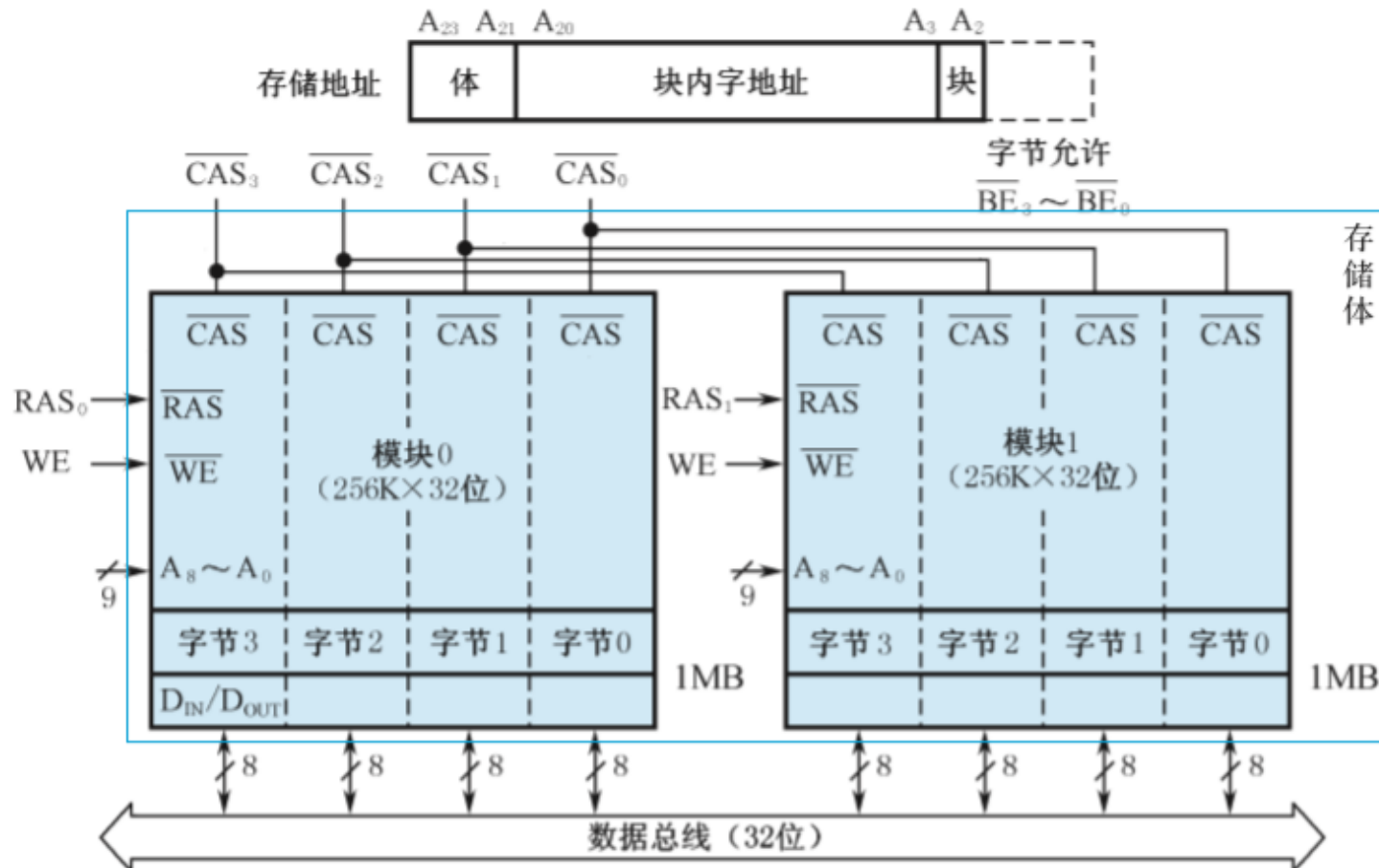
顺序存储器和交叉存储器的带宽分别是：

$$W_2 = q / t_2 = 256b \div (8 \times 10^{-7})s = 320Mb/s$$

$$W_1 = q / t_1 = 256b \div (35 \times 10^{-7})s = 730Mb/s$$

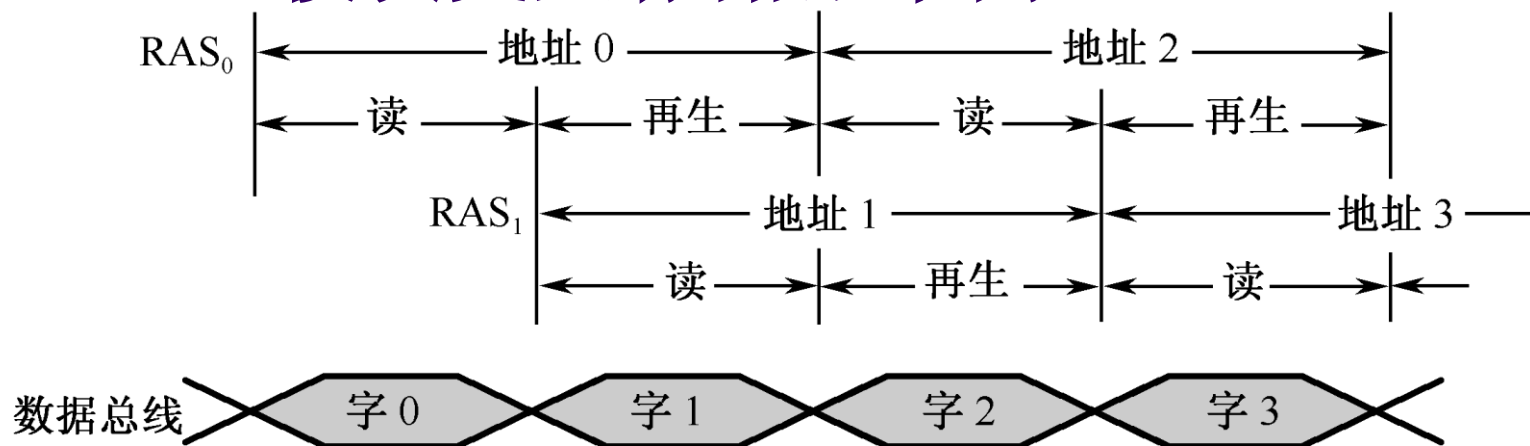
3.5.2 多模块交叉存储器

3.二模块交叉存储器举例



3.5.2 多模块交叉存储器

3.二模块交叉存储器举例



3.6 cache存储器

3.6.1 cache基本原理

3.6.2 主存与cache的地址映射

3.6.3 cache的替换策略

3.6.4 cache的写操作策略

3.6.5 Pentium4的cache组织

3.6.6 使用多级cache减少缺失损失

3.6.1 cache基本原理

1.cache的功能

解决CPU和主存之间的速度不匹配问题

原理基于程序运行中的空间局部性和时间局部性

- 为追求高速，包括管理在内的全部功能由硬件实现，因而对程序员是透明的。
- 随着半导体器件集成度的进一步提高，可以将小容量的 cache 与 CPU 集成到同一芯片中，其工作速度接近于 CPU 的速度，从而组成两级以上的 cache 系统。

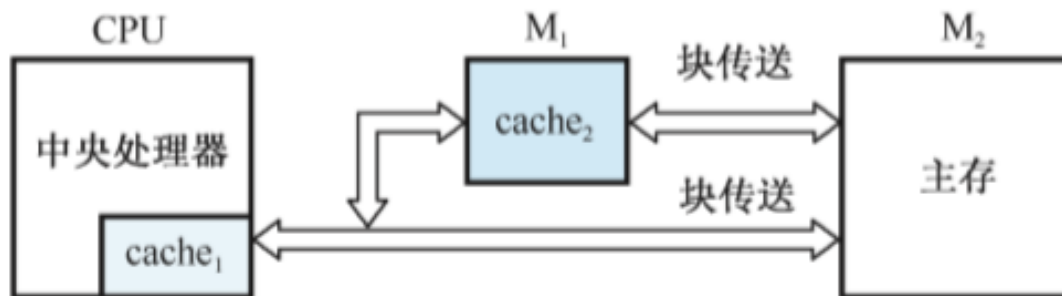
3.6.1 cache基本原理

CPU与存储器系统的关系

如图所示，cache是介于CPU和主存M₂之间的**小容量存储器**，但存取**速度比主存快**，容量远小于主存。

cache 能高速地向**CPU**提供指令和**数据**，从而加快了程序的执行速度。

从功能上看，它是主存的缓冲存储器，由**高速的 SRAM** 组成。



3.6.1 cache基本原理

2.cache基本原理

- cache 除包含 SRAM 外，还要有控制逻辑。
 - 若 cache 在 CPU 外，它的控制逻辑一般与主存控制逻辑合成在一起，称为主存/cache 控制器；
 - 若 cache 在 CPU 内，则由 CPU 提供它的控制逻辑。

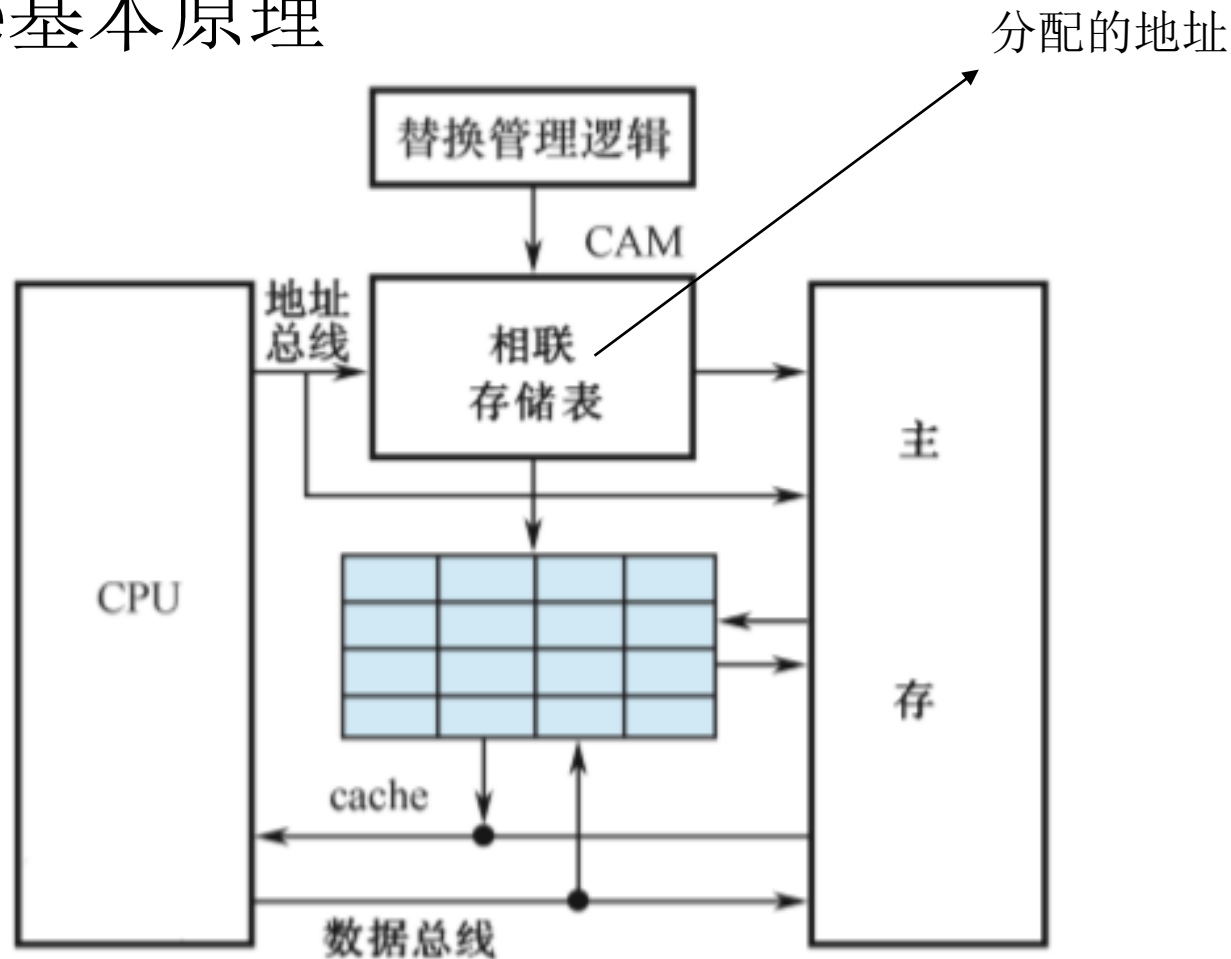
3.6.1 cache基本原理

2.cache基本原理

- CPU与cache之间的数据交换是以字为单位，
- cache 与主存之间的数据交换是以块为单位。
- 一个块由若干字组成，是定长的。
- 当 CPU 读取内存中一个字时，便发出此字的内存地址到 cache 和主存。
 - 此时 cache 控制逻辑依据地址判断此字当前是否在 cache 中：
 - 若是，则 cache 命中，此字立即传送给 CPU；
 - 若非，则 cache 缺失(未命中)，用主存读周期把此字从主存读出送到 CPU，与此同时，把含有这个字的整个数据块从主存读出送到 cache 中。

3.6.1 cache基本原理

2.cache基本原理



3.6.1 cache基本原理

3. Cache的命中率

从CPU来看，增加一个cache的目的，就是在性能上使主存的平均读出时间尽可能接近cache的读出时间。为了达到这个目的，在所有的存储器访问中由cache满足CPU需要的部分应占很高的比例，即cache的命中率应接近于1。由于程序访问的局部性，实现这个目标是可能的。

在一个程序执行期间，设 N_c 表示cache完成存取的总次数， N_m 表示主存完成存取的总次数， h 定义为命中率，则有

$$h = \frac{N_c}{N_c + N_m}$$

3.6.1 cache基本原理

3. Cache的命中率

从CPU来看，增加一个cache的目的，就是在性能上使主存的平均读出时间尽可能接近cache的读出时间。为了达到这个目的，在所有的存储器访问中由cache满足CPU需要的部分应占很高的比例，即cache的命中率应接近于1。由于程序访问的局部性，实现这个目标是可能的。

在一个程序执行期间，设 N_c 表示cache完成存取的总次数， N_m 表示主存完成存取的总次数， h 定义为命中率，则有

$$h = \frac{N_c}{N_c + N_m}$$

3.6.1 cache基本原理

3.cache命中率公式

命中率 $h = \frac{N_e}{N_e + N_m}$

Cache/主存的平均访问时间 $t_a = ht_c + (1-h)t_m$

访问效率 $e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} = \frac{1}{h + (1-h)r} = \frac{1}{r + (1-r)h}$

主存与Cache的访问时间比 $r = t_m / t_c$

例6 CPU执行一段程序时，**cache**完成存取的次数为**1900**次，主存完成存取的次数为**100**次，已知**cache**存取周期为**50ns**，主存存取周期为**250ns**，求**cache/主存**系统的效率和平均访问时间。

解：

- $h = N_c / (N_c + N_m) = 1900 / (1900 + 100) = 0.95$
- $t_a = h * t_c + (1 - h) * t_m = 0.95 * 50 + 0.05 * 250 = 60\text{ns}$
- $e = t_c / t_a = 50 / 60 = 83.3\%$

3.6.1 cache基本原理

4.cache 结构设计必须解决的问题

- cache 的设计需要遵循两个原则：
- cache 的**命中率**尽可能高，实际应接近于 1；
- cache 对 CPU 而言是**透明**的，即不论是否有 cache，CPU 访存的方法都是一样的，软件不需增加任何指令就可以访问 cache。

解决了命中率和透明性问题，就 CPU 访存的角度而言，内存将具有主存的容量和接近 cache 的速度。为此，必须增加一定的硬件电路完成控制功能，即 **cache 控制器**。

在设计 cache 结构时，必须解决后面4个问题。

3.6.1 cache基本原理

4.cache 结构设计必须解决的问题

- ①主存的内容调入 cache 时如何存放？
- ②CPU访存时如何找到 cache 中的信息？

前两个问题是相互关联的，即如何将主存信息定位在 cache 中，如何将主存地址变换为 cache 地址。与主存容量相比，cache 的容量很小，它保存的内容只是主存内容的一个子集，且 cache 与主存的数据交换是以块为单位。

为了把主存块放到 cache 中，必须应用某种方法把主存地址定位到 cache 中，称为地址映射。“映射”一词的物理含义是确定位置的对应关系，并用硬件来实现。

这样当 CPU 访问存储器时，它所给出的一个字的内存地址会自动变换成 cache 的地址，即 cache 地址变换。

3.6.1 cache基本原理

4.cache 结构设计必须解决的问题

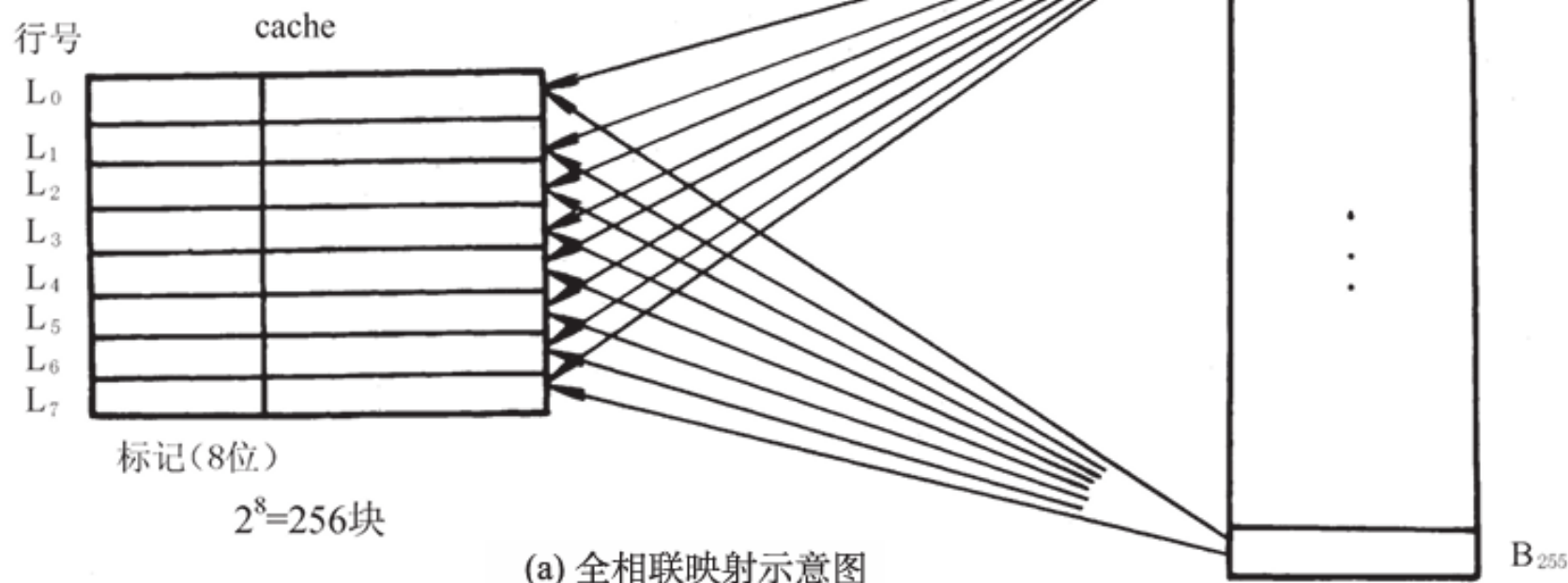
- ③当cache空间不足时如何替换 cache 中已有的内容
 - cache 替换问题主要是选择和执行替换算法，以便在 cache 不命中时替换 cache 中的内容。
- ④需要写操作时如何改写 cache 的内容？
 - 涉及 cache 的写操作策略，重点是在更新时保持主存与 cache 的一致性。

3.6.2 主存与Cache的地址映射

- 以下我们介绍三种映射方法
 - 全相联方式
 - 直接方式
 - 组相联方式

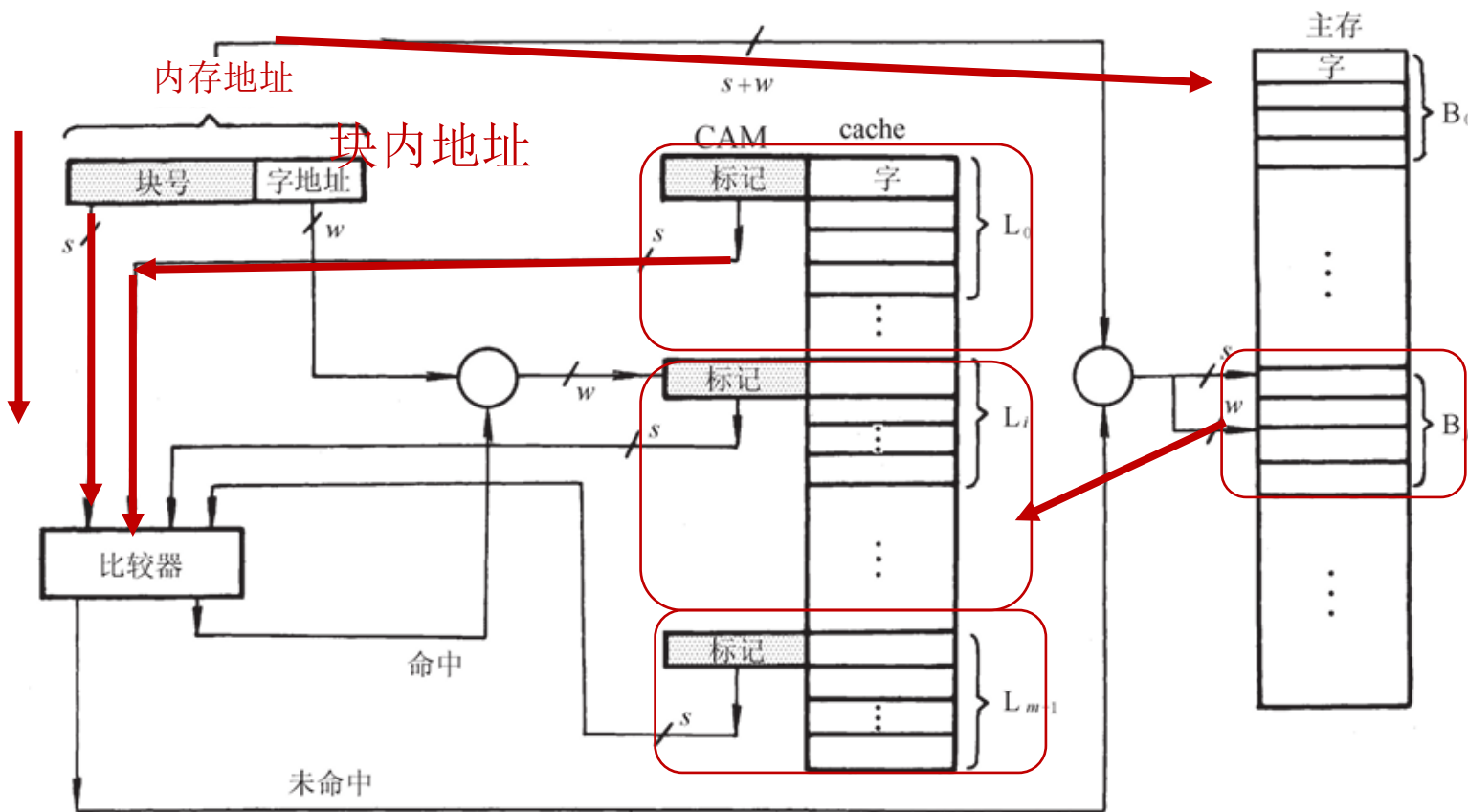
1.全相联的映射方式

主存地址长度= $(s+w)$ 位, 寻址单元数= 2^{s+w} 个字
块大小=行大小= 2^w 个字, 主存的块数= 2^s
标记位数= s 位, cache的行数=不由地址格式确定



(a) 全相联映射示意图

1.全相联的映射方式



(b) 全相联cache的检索过程

3.6.2 主存与cache的地址映射

1. 全相联的映射方式

转换公式

主存地址长度 = $(s+w)$ 位

寻址单元数 = 2^w 个字或字节

块大小 = 行大小 = 2^w 个字或字节

主存的块数 = 2^s

标记大小 = s 位

cache的行数 = 不由地址格式确定

3.6.2 主存与cache的地址映射

例题：内存空间为1K个单元，每四个单元划分为1块，cache中有8行，每一行可以放入内存中的一个块。Cache与主存采用全相联映射方式。请问：

- (1) tag有多少位？
- (2) 将主存中的0号、1号、8号、9号放入cache怎么放置？

3.6.2 主存与cache的地址映射

1.全相联的映射方式

特点：

- 优点：冲突概率小，Cache的利用高。
- 缺点：高速比较器难实现，需要一个访问速度很快
代价高的相联存储器

应用场合：

- 适用于小容量的Cache

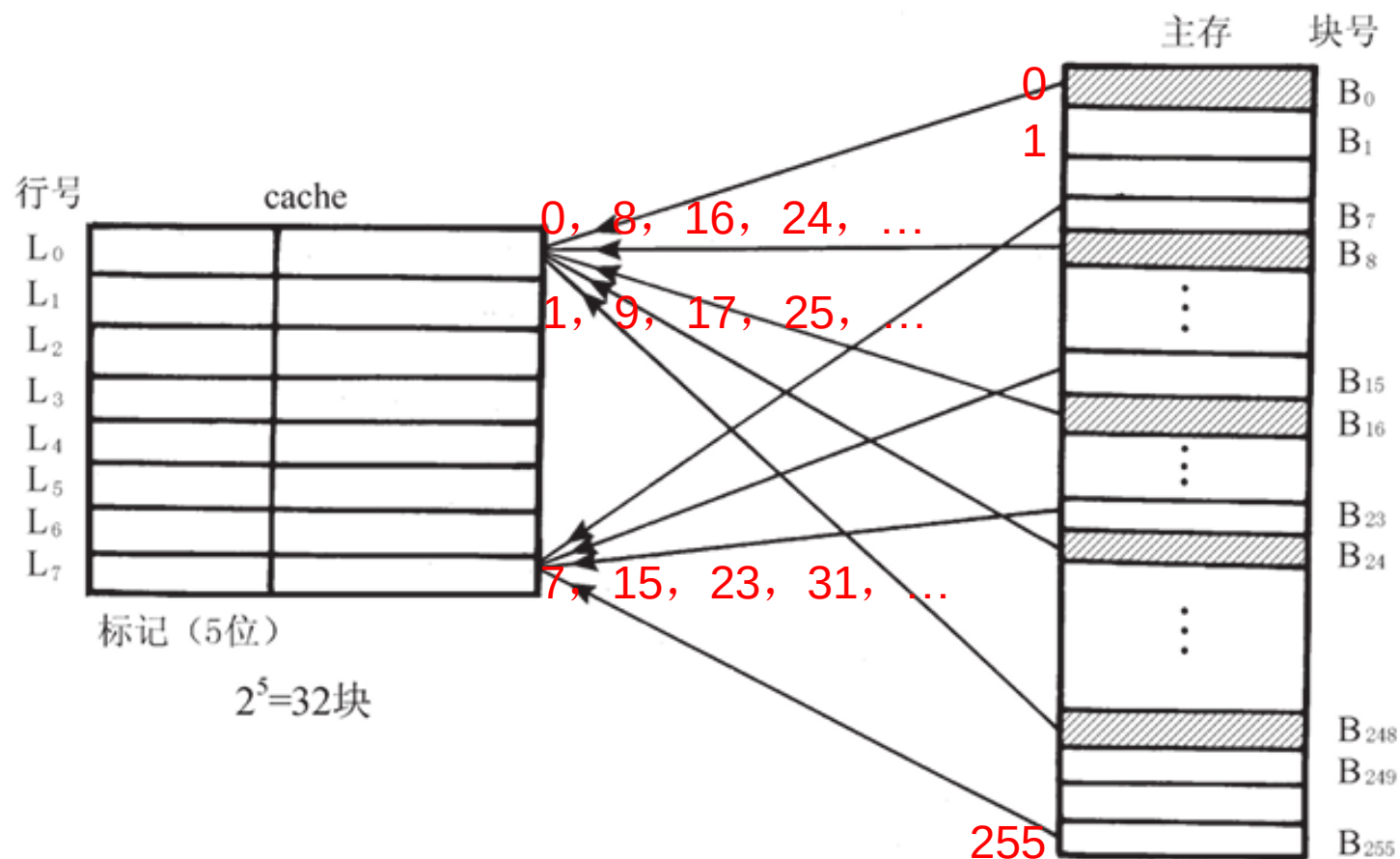
3.6.2 主存与cache的地址映射

2.直接映射方式（多对一的映射关系）

Cache的行号*i*和主存的块号*j*的映射方法如：

- $i = j \bmod m$
- *m*为cache中总行数

2.直接映射方式



(b) 直接映射示意图

3.6.2 主存与cache的地址映射

2.直接映射方式

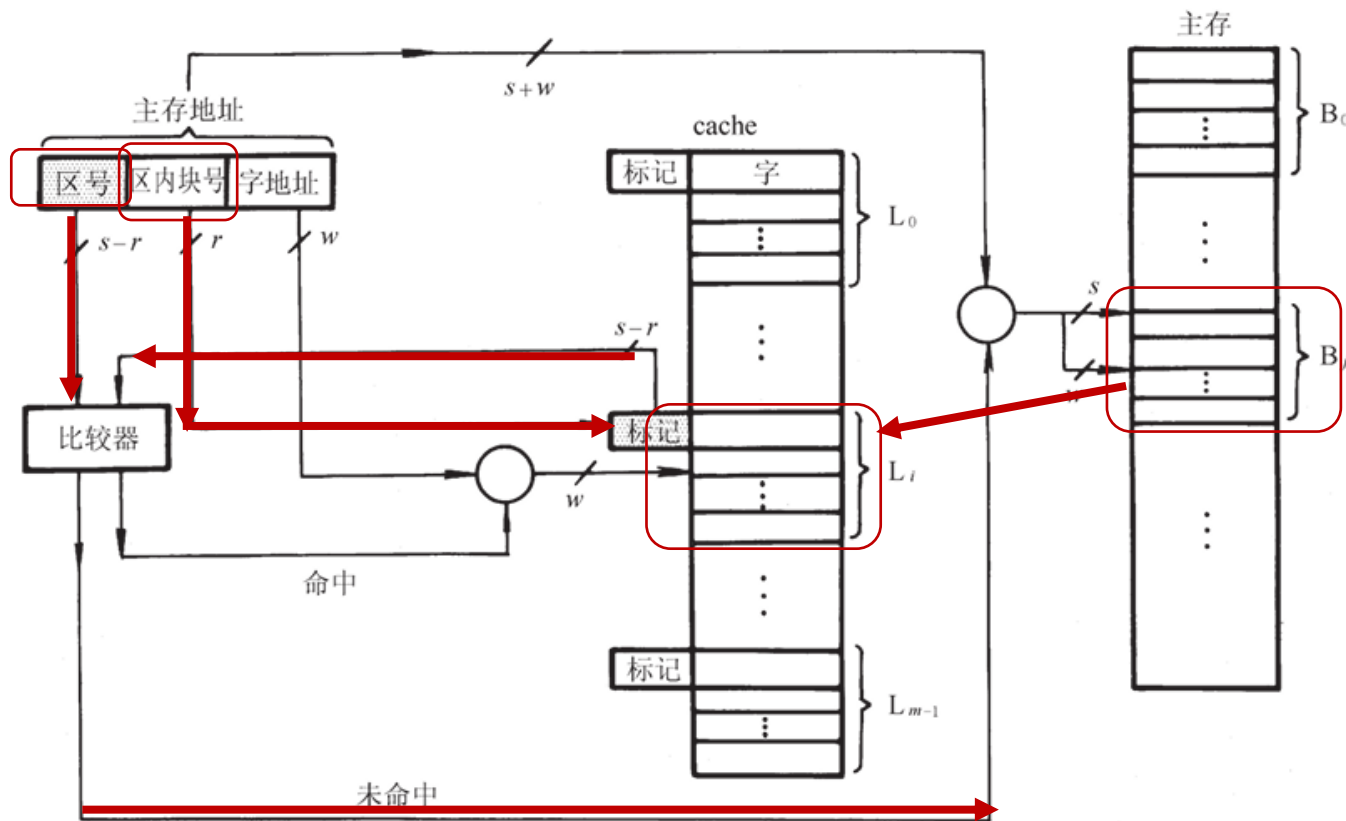
特点

- 优点：硬件简单，成本低，地址变换速度快。
- 缺点：每个主存块只有一个固定的行位置可存放。如果连续访问块号相距 m 整数倍的两个块，因两个块映射到同一 **cache** 行时，就会发生冲突。发生冲突时就要将原先存入的行换出去，但很可能过一段时间又要换入。频繁的置换会使 **cache** 效率下降。

应用场合

- 适合大容量Cache，更多的行数可以减小冲突的机会。

2.直接映射方式



(c) 直接映射cache的检索过程

图 3.31 直接映射的 cache 组织

3.6.2 主存与cache的地址映射

2. 直接映射方式

转换公式

主存地址长度 = $(s+w)$ 位

寻址单元数 = 2^{s+w} 个字或字节

块大小 = 行大小 = 2^w 个字或字节

主存的块数 = 2^s

cache的行数 = $m = 2^r$

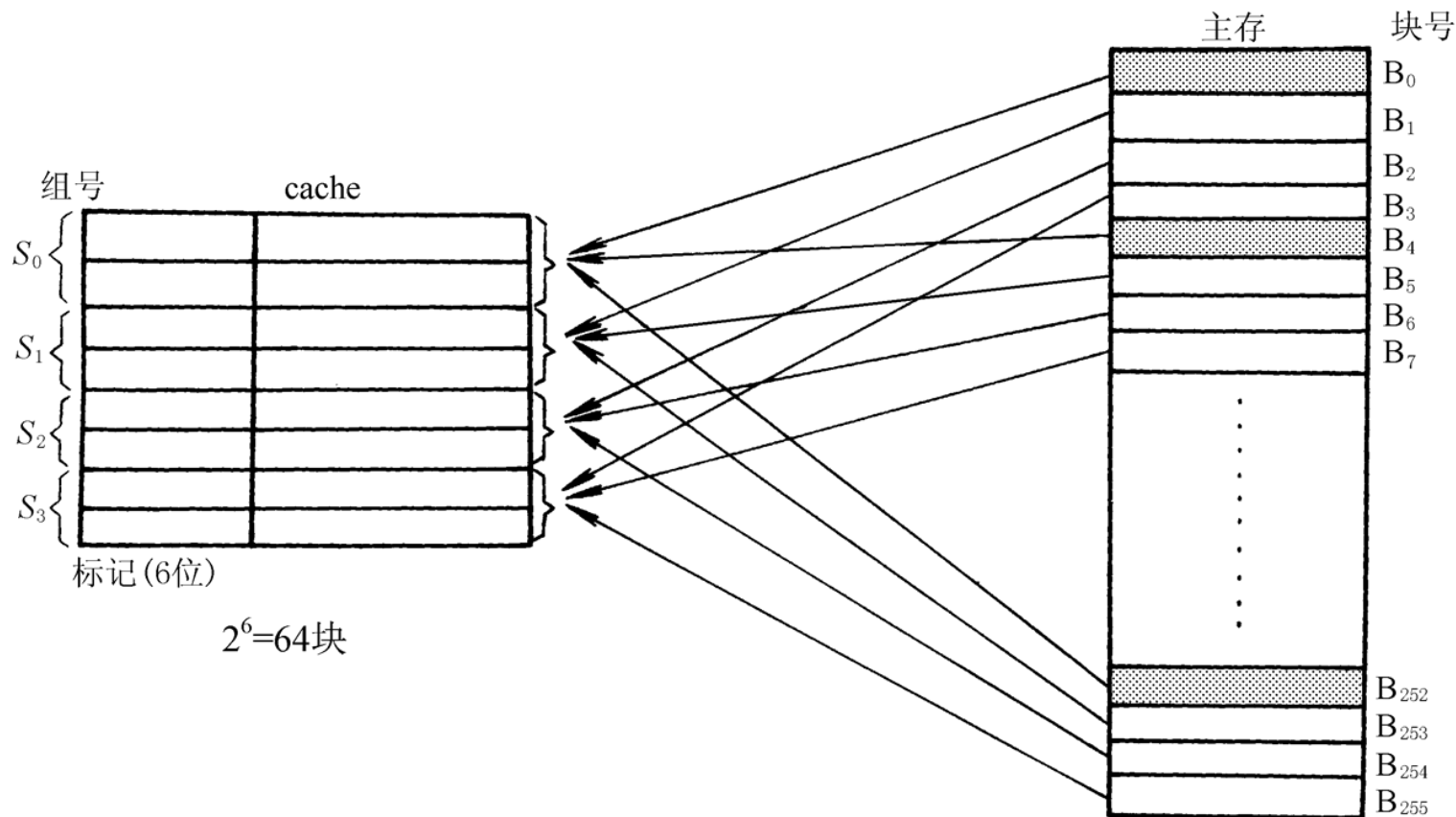
标记大小 = $(s-r)$ 位

3.6.2 主存与cache的地址映射

3.组相联映射方式

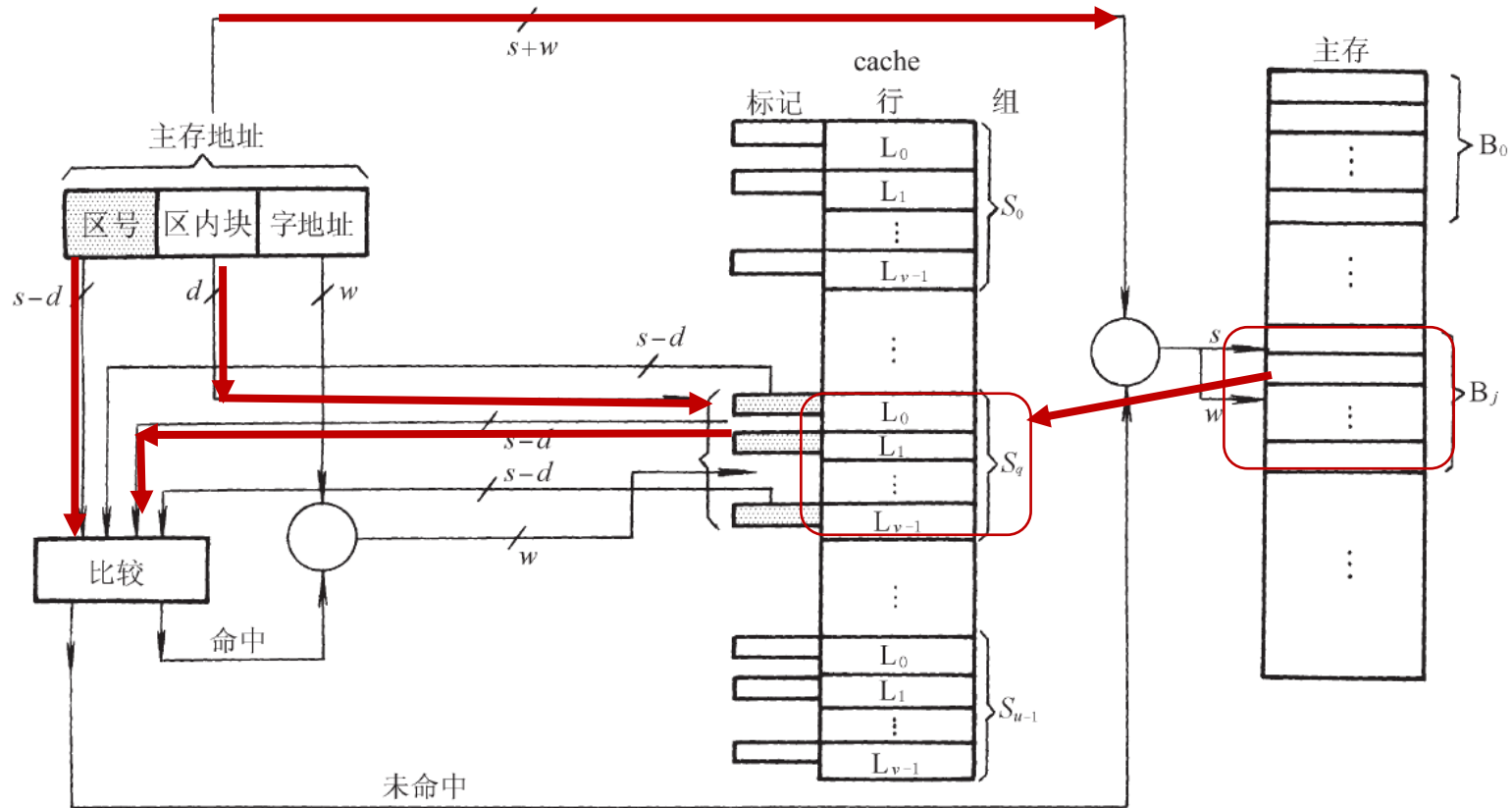
- 前两者的组合
 - Cache分组，组间采用直接映射方式，组内采用全相联的映射方式
 - Cache分 u 组，每组 v 行（ v 路组相联）： $m=u \times v$
 - 映射方法（一对多）
 - 组号 $q = j \bmod u$
 - 主存第 j 块内容拷贝到Cache的 q 组中的某行

3.组相联映射方式



- 优点：冲突小于直接映射，硬件复杂程度小于全相联映射。

3.组相联映射方式



(c) 组相联cache的检索过程

图 3.32 组相联映射的 cache 组织

3.6.2 主存与cache的地址映射

3.组相联映射方式

转换公式

主存地址长度 = $(s+w)$ 位

寻址单元数 = 2^{s+w} 个字或字节

块大小 = 行大小 = 2^w 个字或字节

主存的块数 = 2^s

每组的行数 = v

组数 $u = 2^d$

cache的行数 = kv

标记大小 = $(s-d)$ 位

3.6.2 主存与cache的地址映射

例9：有一个处理器，主存容量 1MB，字长 1B，块大小 16B，cache 容量 64KB。若 cache 采用全相联映射，对内存地址(B0010)16给出相应的标记和字地址。

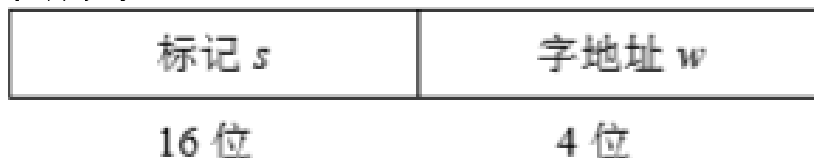
解：

块大小=行大小= 2^4 字节= 2^w 字节 $\therefore w=4$

主存寻址单元数= $2^{s+w}=1M=2^{20}$ $\therefore s+w=20$, $s=16$ 位

标记大小= $s=16$ 位

内存地址格式如下所示：



由于内存地址(B0010)₁₆=(1011 0000 0000 0001 0000)₂

故对应的标记 $s=(1011 0000 0000 0001)_2$ 字地址 $w=(0000)_2$

3.6.2 主存与cache的地址映射

例8：一个组相联cache由64个行组成，每组4行。主存包含4K个块，每块128字。请表示内存地址的格式。

解：块大小=行大小= 2^w 个字= $128=2^7$ $\therefore w=7$

每组的行数= $v=4$

组数 $u=2^d=2^4=16$ $\therefore d=4$

主存的块数= $2^s=4K=2^2 \times 2^{10}=2^{12}$ $\therefore s=12$

标记大小= $s-d=12-4=8$ (位)

主存地址长度= $s+w=12+7=19$ (位)

主存寻址单元数= $2^{s+w}=2^{19}$

故 $v=4$ 各组相联的内存地址格式如下所示：

标记s-d	组号d	字号w
8位	4位	7位

3.6.3 cache的替换策略

- **LFU**（最不经常使用）：被访问的行计数器增加**1**，换出值最小的行同时将行的计数器清零。
- **LRU**（近期最少使用）：被访问的行计数器置**0**，其他计数器增加**1**，换出值最大的行，保护了刚复制到**cache**中的新数据行，符合**cache**的工作原理，使**cache**有较高的命中率。
- **随机替换**：随机替换策略实际上是不要什么算法，从特定的行位置中随机地选取一行换出即可。
 - 在硬件上容易实现，且速度也比前两种策略快。
 - 缺点是随意换出的数据很可能马上又要使用，从而降低命中率和**cache**工作效率。但这个不足随着**cache**容量增大而减小。随机替换策略的功效只是稍逊于前两种策略。

3.6.4 cache的写操作策略

- 由于cache的内容只是主存部分内容的拷贝，它应当与主存内容保持一致。而CPU对cache的写入更改了cache的内容。如何与主存内容保持一致，可选用如下三种写操作策略。
 - **写回法**：当CPU写cache命中时，只修改cache的内容，而不立即写入主存；只有当此行被换出时才写回主存。

3.6.4 cache的写操作策略

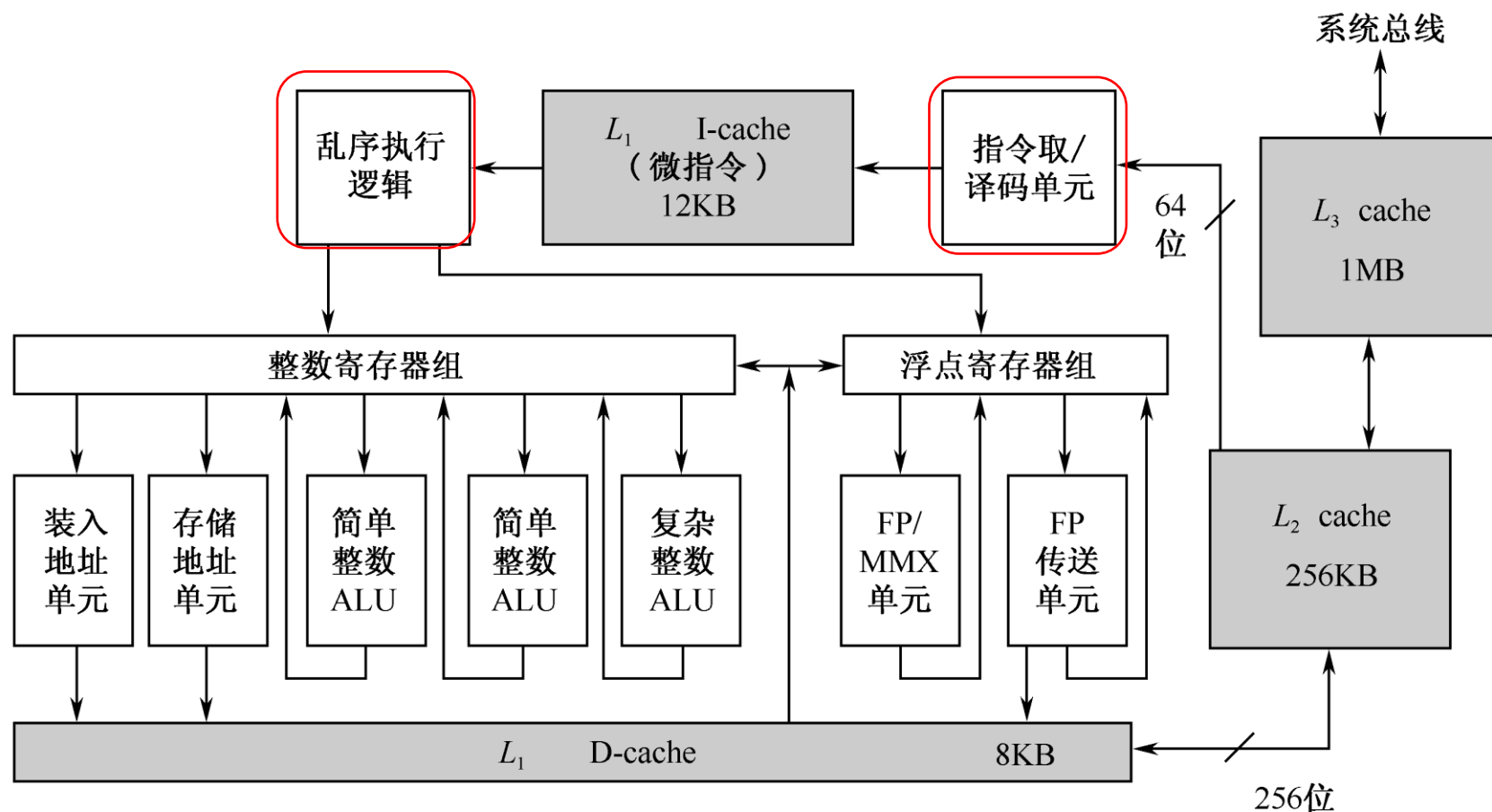
- 由于cache的内容只是主存部分内容的拷贝，它应当与主存内容保持一致。而CPU对cache的写入更改了cache的内容。如何与主存内容保持一致，可选用如下三种写操作策略。
 - **写回法**：当CPU写cache命中时，只修改cache的内容，而不立即写入主存；只有当此行被换出时才写回主存。
 - **全写法**：写命中时，Cache与内存一起写

3.6.4 cache的写操作策略

- 由于cache的内容只是主存部分内容的拷贝，它应当与主存内容保持一致。而CPU对cache的写入更改了cache的内容。如何与主存内容保持一致，可选用如下三种写操作策略。
 - **写回法**：当CPU写cache命中时，只修改cache的内容，而不立即写入主存；只有当此行被换出时才写回主存。
 - **全写法**：写命中时，Cache与内存一起写
 - **写一次法**：写命中与写未命中的处理方法和写回法基本相同，只是第一次写命中时要同时写入主存。

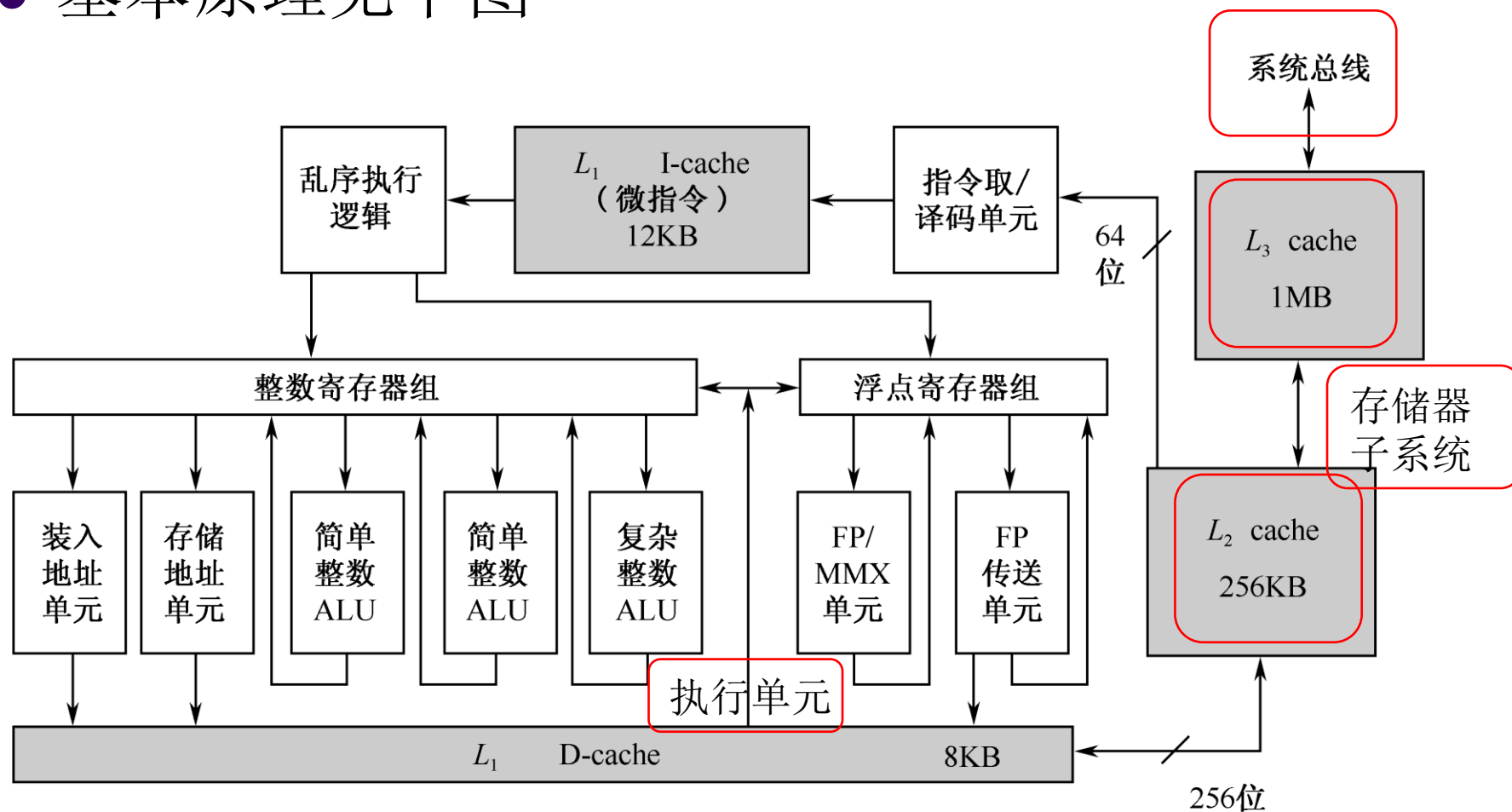
3.6.5 Pentium 4的Cache组织

- 基本原理见下图



3.6.5 Pentium 4的Cache组织

- 基本原理见下图



3.6.6 使用多级cache减少缺失损失

cache 和组成CPU 的微处理器集成到一个芯片上。

为进一步缩小现代CPU和DRAM访问速度的差距，CPU支持附加一级的cache。二级cache在访问主cache缺失时被访问，各级cache都不包含所访问数据时，需要访问主存储器。

[例10] 现有一处理器，基本CPI（每条指令周期数）为1.0，所有访问在第一级cache中命中，时钟频率5GHz。假定访问一次主存储器的时间为100ns，其中包括所有缺失处理。设平均每条指令在第一级cache中产生的缺失率为2%。若增加一个二级cache，命中或缺失的访问时间都为5ns，且容量大到可使必须访问主存的缺失率降为0.5%，问处理器速度提高多少。



[例3.8] 现有一处理器，基本CPI为1.0，所有访问在第一级cache中命中，时钟频率5GHz。假定访问一次主存储器的时间为100ns，其中包括所有缺失处理。设平均每条指令在第一级cache中产生的缺失率为2%。若增加一个二级cache，命中或缺失的访问时间都为5ns，且容量大到可使必须访问主存的缺失率降为0.5%，问处理器速度提高多少。

解 必须访问主存储器的缺失损失为

$$\frac{100\text{ns}}{0.2\text{ns/时钟周期}} = 500\text{个时钟周期}$$

只有一级 cache 的机器有效 CPI 由下式给出：

$$\text{总的 CPI} = \text{基本 CPI} + \text{每条指令中存储器停顿的时钟周期}$$

对只有一级 cache 的本例中处理器来说：

$$\text{总的 CPI} = 1.0 + \text{每条指令存储器停顿的时钟周期} = 1.0 + 2\% \times 500 = 11.0$$

对二级 cache，主 cache（第一级 cache）中发生缺失后可以被第二级 cache 或者主存处理。访问第二级 cache 的缺失损失为

$$\frac{5\text{ns}}{0.2\text{ns / 时钟周期}} = 25\text{个时钟周期}$$

如果第二级 cache 能处理全部缺失，那么这就是整个的缺失损失。如果缺失要求访问主存储器，那么总的缺失损失为访问第二级 cache 和访问主存储器的时间之和。

因此，对一个二级的 cache 而言，总的 CPI 为基本 CPI 和两级 cache 停顿的时钟周期之和：

$$\begin{aligned} \text{总的 CPI} &= 1 + \text{每条指令的一级停顿} + \text{每条指令的二级停顿} \\ &= 1 + 2\% \times 25 + 0.5\% \times 500 = 1 + 0.5 + 2.5 = 4.0 \end{aligned}$$

因此，有二级 cache 的处理器性能是没有二级 cache 的处理器性能的 2.8 倍，即

$$11.0 \div 4.0 \approx 2.8$$

本章小结

- 分类、多级存储体系结构、编址和端模式、存储器的技术指标有**存储容量**、存取时间、存储周期、存储器带宽。
- **SRAM**: 存储元阵列、逻辑结构、读写时序、扩展
- **DRAM**: 工作原理、逻辑结构、读写时序、刷新、突发、**SDRAM**、DDR SDRAM、CDRAM
- 只读存储器
- 并行存储器: **双端口**、交叉存储器
- **Cache**: 工作原理、命中率、地址映射、替换策略、写操作、pentium4、减少缺失损失

作业



1. 第三章课后题1、2、4、8、9、10、13、15
2. 等后面内容学完布置其他题再一起交